

Behavior Trees in Video Games

Marvin Maerevoet

h_da

768036

Advanced Game AI



Table of Contents

- Intro
- BT Introduction
- Approaches & Tools
- Examples in Games
- Limitations
- Elegance
- Conclusion



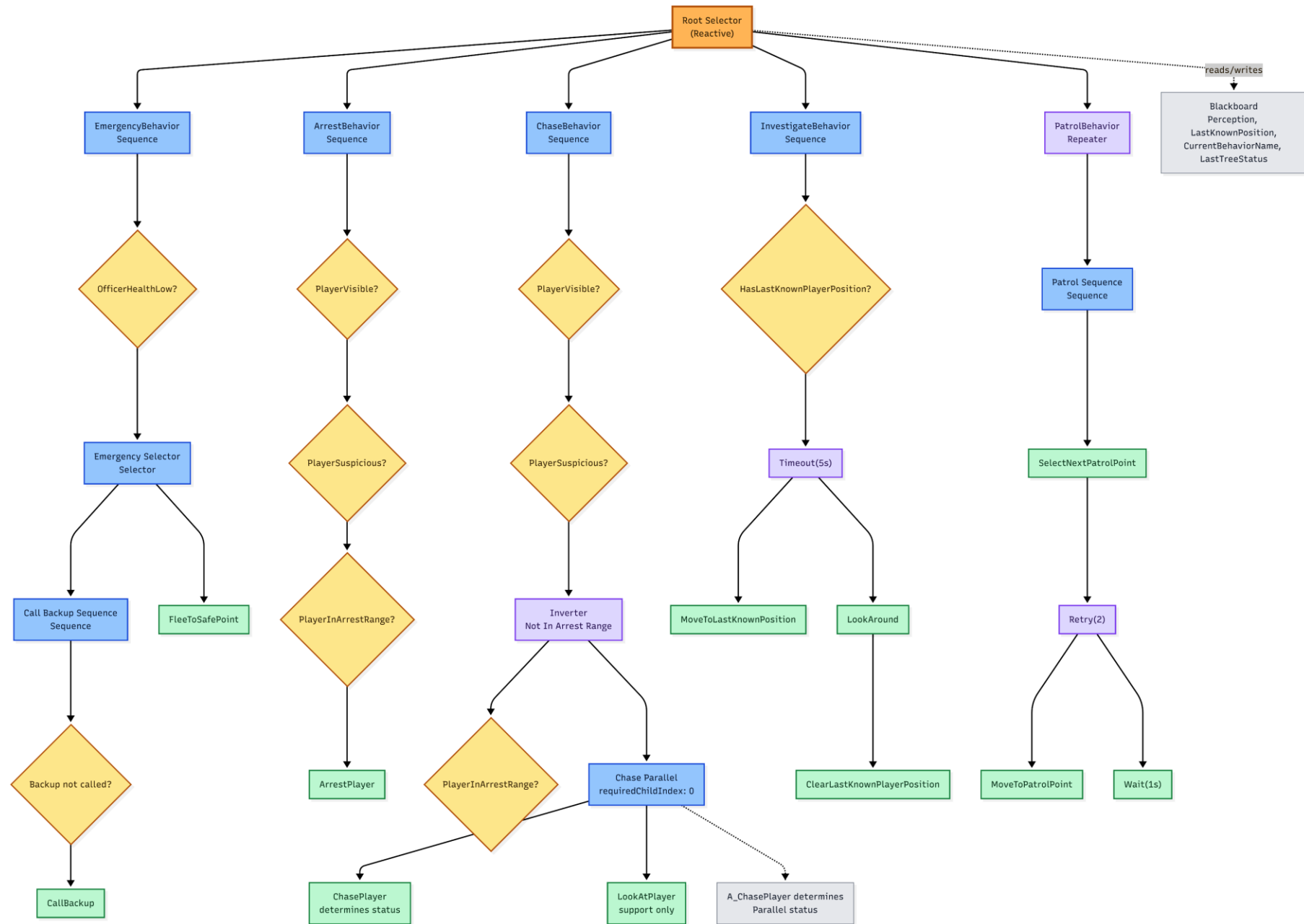


Why Behavior Trees?

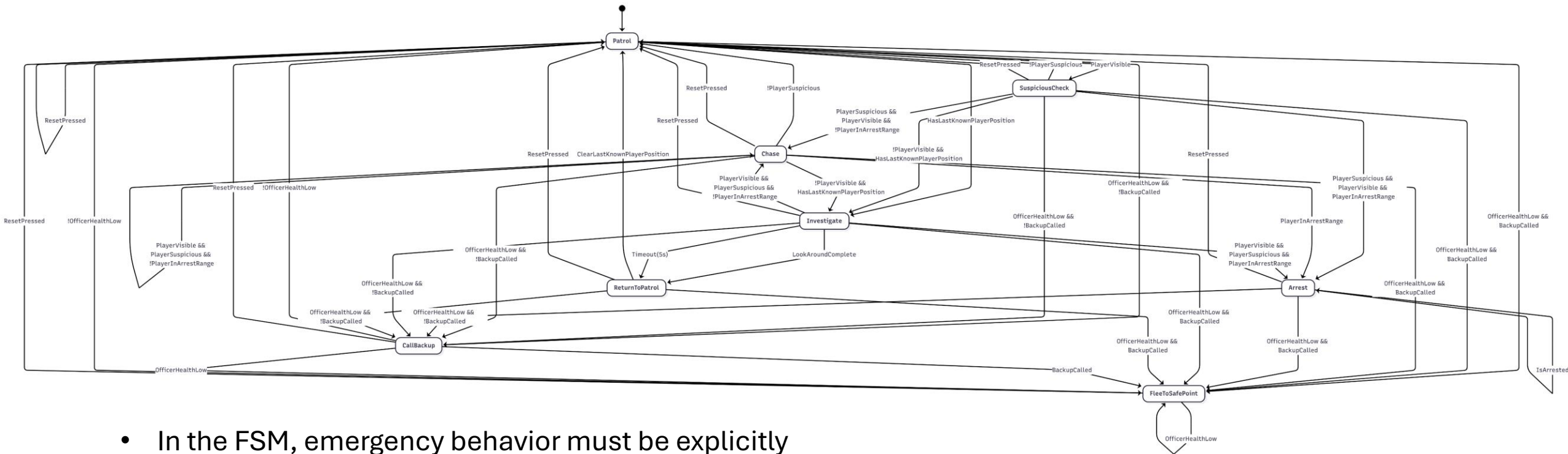
- We already know FSMs
- Why do we need a new concept?
- Answer: designer-friendly structure

Scenario

- A police officer patrols.
- The police officer sees the player.
- The police officer approaches and talks to them.
- If the player runs away: chase them.
- If the player is too far away: investigate their last known position.
- If the player is close enough: arrest/attack them.
- If the police officer is injured: retreat or call for backup.



Equivalent FSM Diagram



- In the FSM, emergency behavior must be explicitly modeled as transitions from almost every state.
- In the Behavior Tree, one prioritized emergency branch in the Reactive Root Selector is enough.

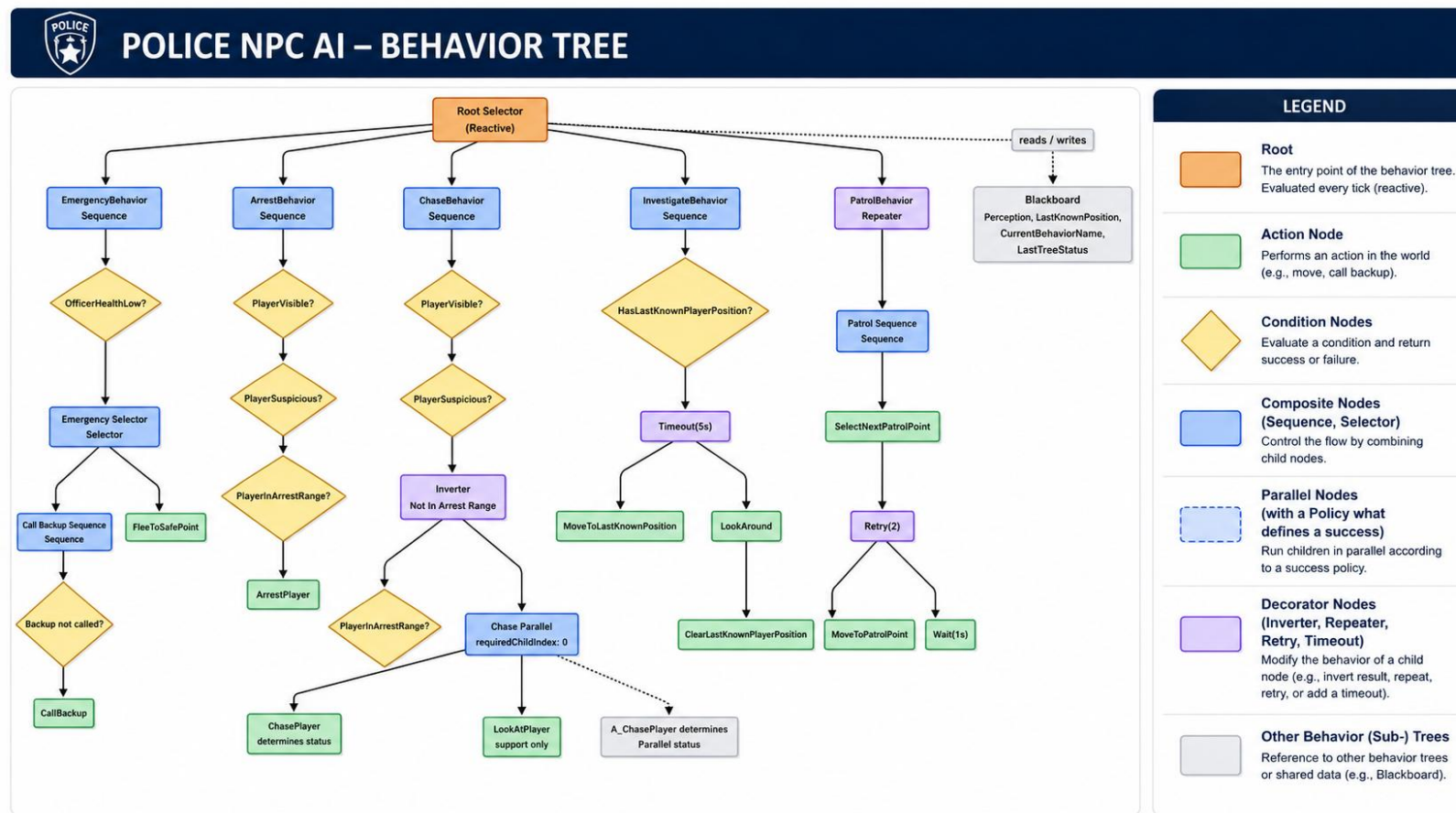
Behavior Tree Components

- A tree consists of nodes
- Nodes have a status {success, failure, running}
- Important node types
 - Root
 - Action Node
 - Condition Nodes
 - Composite Nodes (Sequence, Selector)
 - Parallel Nodes (with a policy that defines success)
 - Decorator Nodes (Inverter, Repeater, Retry, Timeout)

Behavior Tree Components

- Other Behavior (Sub-) Trees
- Blackboard (a shared storage for information)
- Condition and action nodes write to the Blackboard
- Ticking (tree is executed every tick, every 10 ms, or every frame)
- Reactive BT (executes the whole tree every tick)
- Stateful BT (remembers the running node)

Behavior Tree Components



Two Types of Behavior Trees

Reactive BT

- Runs the full tree every tick
- **Advantage:** Reacts quickly to new priorities.
- **Disadvantage:** Running actions can be interrupted constantly.

Stateful BT

- The tree remembers running nodes
- **Advantage:** More stable behavior.
- **Disadvantage:** May react more slowly to new threats.

Pros and Cons of Behavior Trees

Advantages

- Modular behavior structure
- Clear priorities
- Easy to read
- Easier to debug with visualization
- Reusable nodes
- Better structure for complex AI than plain FSMs

Disadvantages

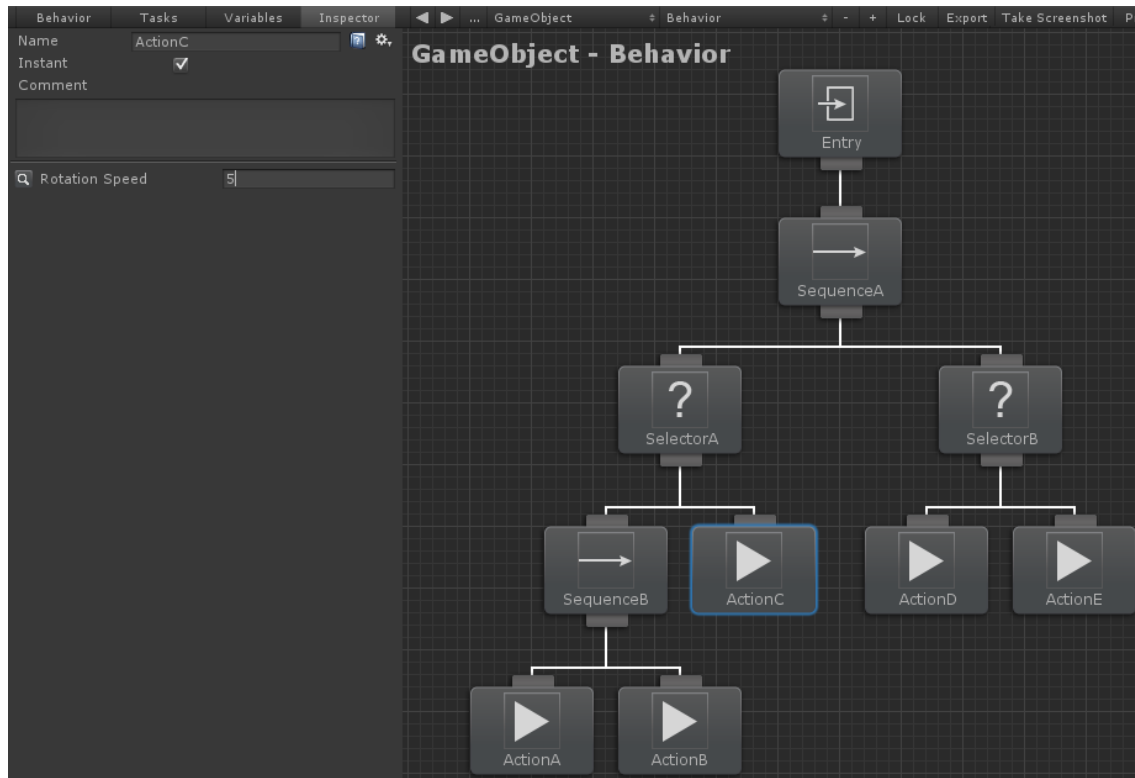
- Large trees can become hard to read
- Blackboard data can become messy
- Complex parallel behavior is difficult
- Hard to debug without good tools
- Logic can be spread across many nodes
- Not ideal for strongly planning-based AI

Approaches and Tools

- Unreal Engine, Behavior Designer, NodeCanvas, Game Creator, and others define their own concrete semantics
- Selector, Sequence, Parallel, Decorator, Blackboard, and tick behavior can work differently depending on the tool
- There are **many de facto standards**
- There is **no single official Behavior Tree standard**

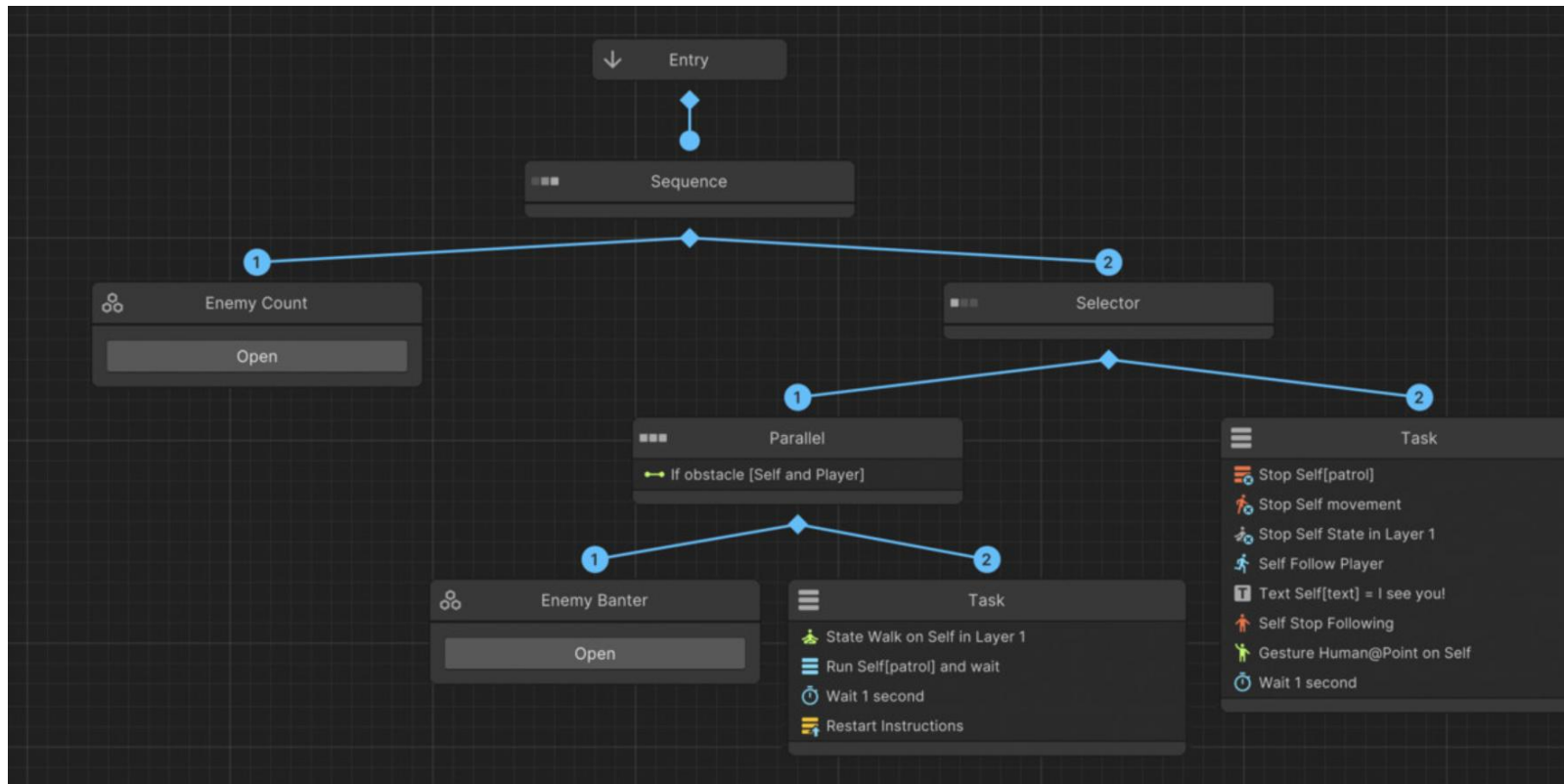
Approaches and Tools

- Unity Behavior Designer



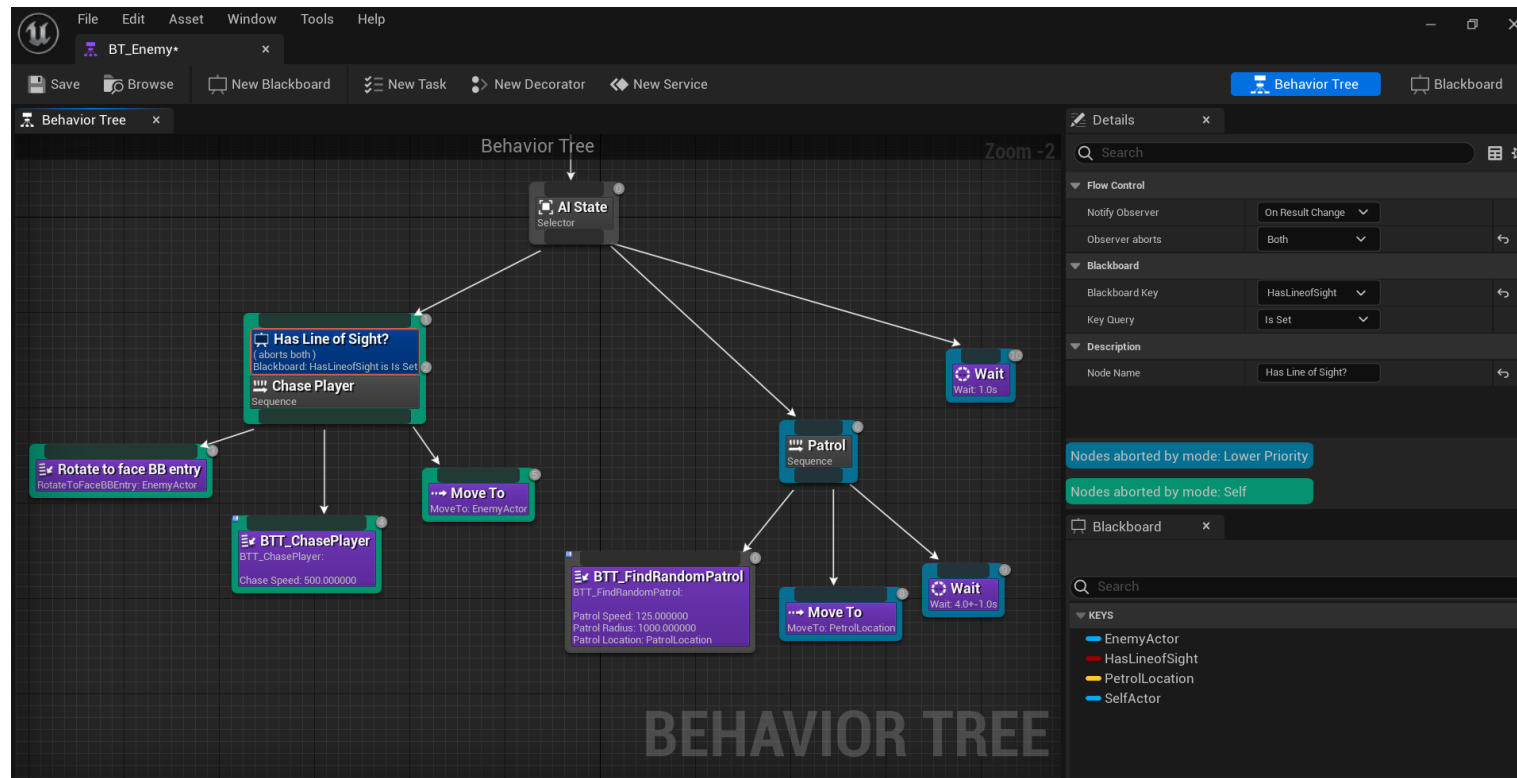
Approaches and Tools

- Game Creator 2: Behavior 2



Approaches and Tools

- Behavior Tree in Unreal Engine



Examples in Games

Famous examples:

- Halo 2 — enemy combat AI: Elites and Grunts use behavior DAGs similar to Behavior Trees for patrol, cover, attack, retreat, and grenade reactions
- BioShock — enemy/NPC AI: Splicers and Big Daddies are often cited as high-profile game AI examples using Behavior Trees
- Spore — creature AI: modular creature behavior was structured with behavior-tree-style logic.

Examples in Games

Examples from our list:

- Skylanders: Imaginators — likely used for boss/enemy combat behavior, such as multi-stage boss behavior or enemy encounter logic
- Starlink: Battle for Atlas — likely used for AI behavior / enemy or ship decision logic

Examples in Games

Conceptual examples:

- Boss fights in Soulslike games
- Patrolling enemies in games like Far Cry

Limitations vs GOAP

What is GOAP

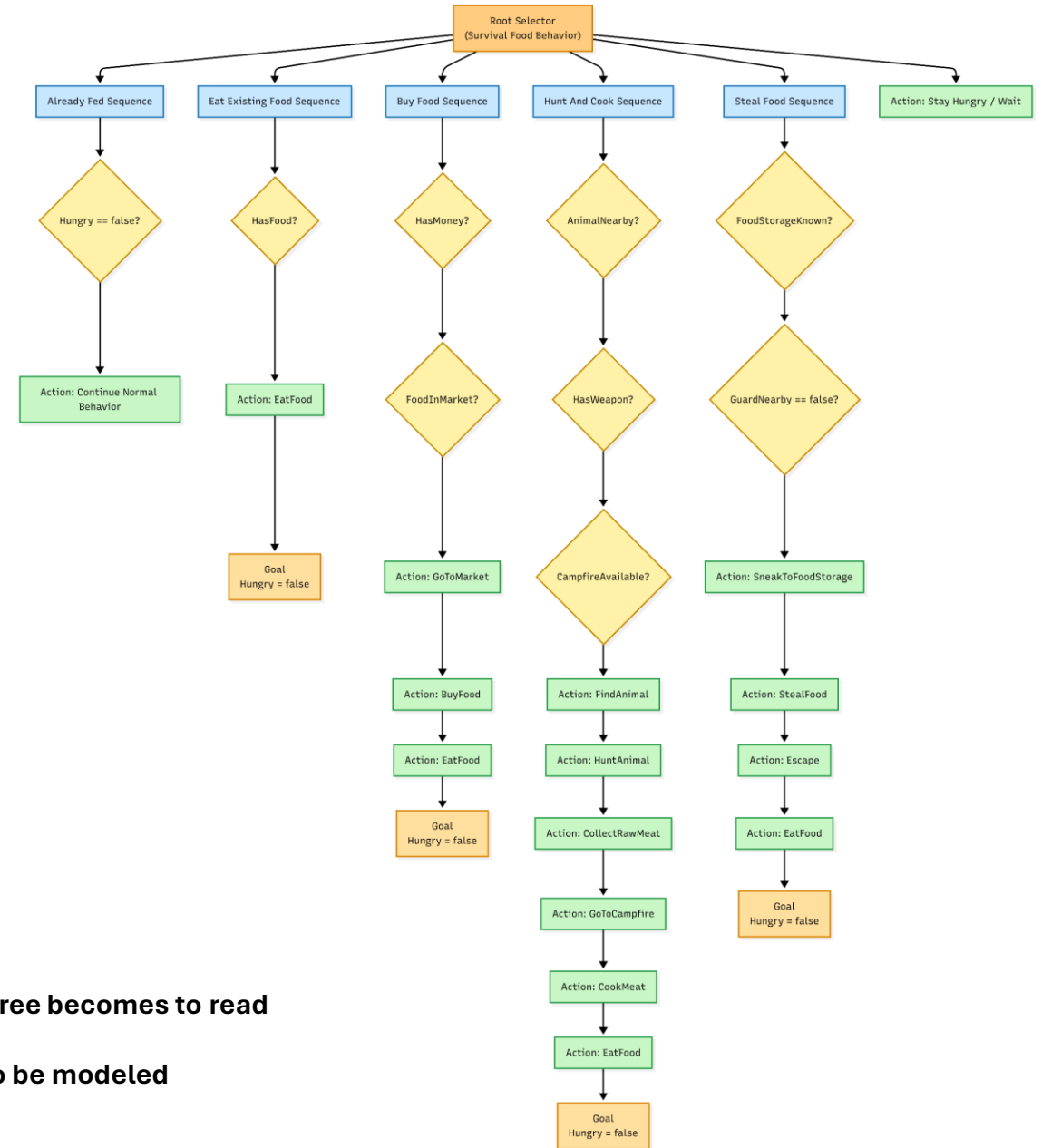
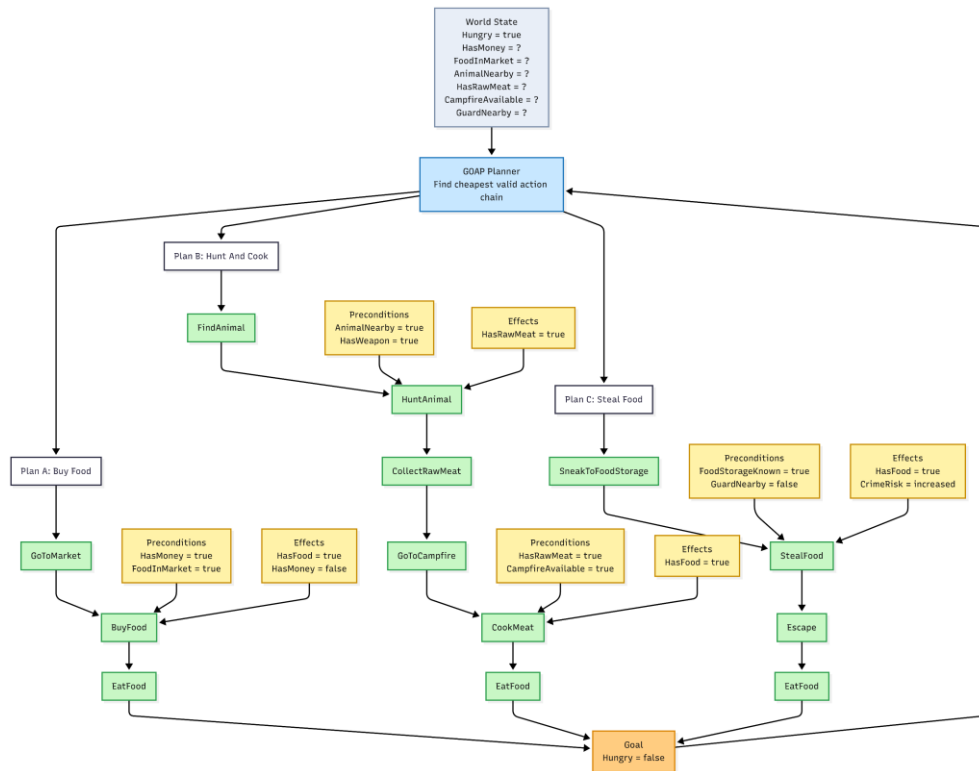
- AI chooses actions by planning from the current world state toward a goal
- Useful when there are many possible ways to solve a problem

Limitations vs GOAP

Scenario

- A hungry survival NPC needs food in a dynamic world
- GOAP can plan different valid paths: buy, hunt, cook, gather, or steal
- A Behavior Tree would become large and repetitive because every alternative path must be manually wired

Limitations vs GOAP



The more alternative paths exist toward the same goal, the harder a Behavior Tree becomes to read

GOAP stays compact because only actions, preconditions, and effects need to be modeled

Limitations vs Utility AI

What is Utility AI

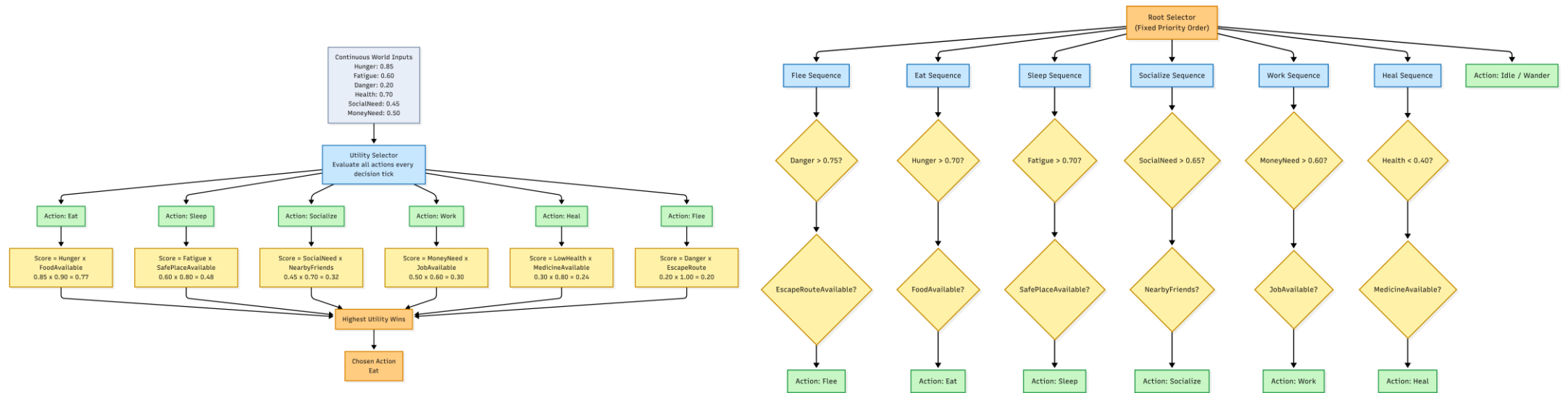
- AI scores possible actions and picks the one with the highest value
- Useful when many needs compete at the same time

Limitations vs Utility AI

Scenario

- A village NPC chooses between needs like hunger, fatigue, danger, health, socializing, and work.
- Utility AI scores all possible actions and selects the best one at that moment.
- A Behavior Tree can feel rigid because fixed priorities may create unnatural decisions.

Limitations vs Utility AI



The more competing options and influencing factors exist, the harder a Behavior Tree becomes to read.

Utility AI stays compact because actions are simply scored.



Why Use Behavior Trees Then?

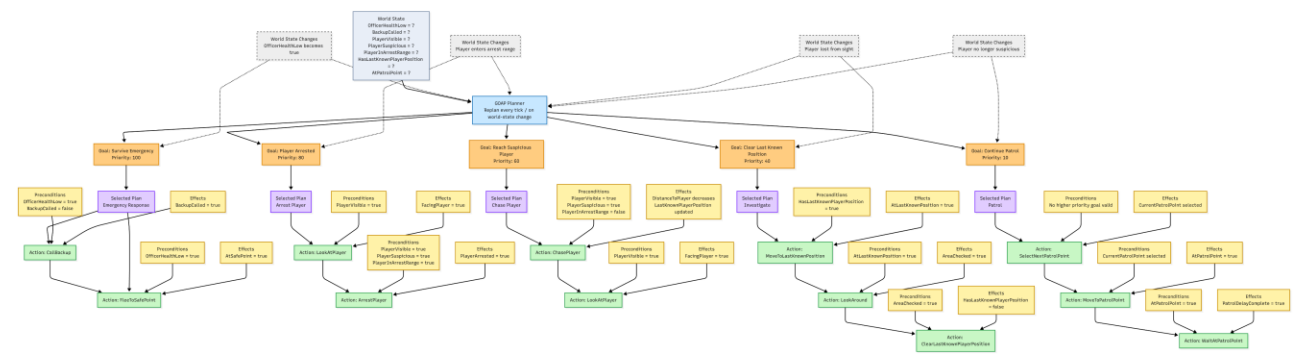
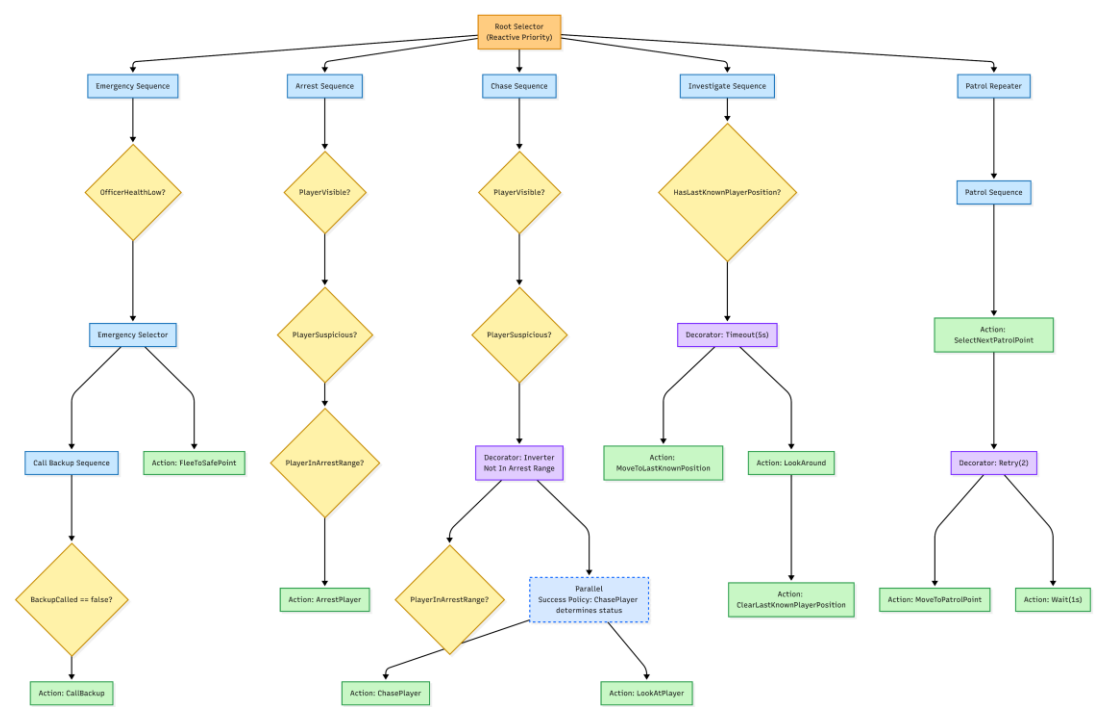
- If GOAP and Utility AI are so useful, why learn Behavior Trees?
- Because in some scenarios, Behavior Trees are more elegant

Advantages vs GOAP

Scenario

- A guard NPC follows a clear routine: patrol, detect, chase, arrest, investigate, and return.
- A Behavior Tree fits because priorities and reactive interruptions are easy to structure.
- GOAP would add unnecessary planning overhead without making this fixed routine clearer.

Advantages vs GOAP

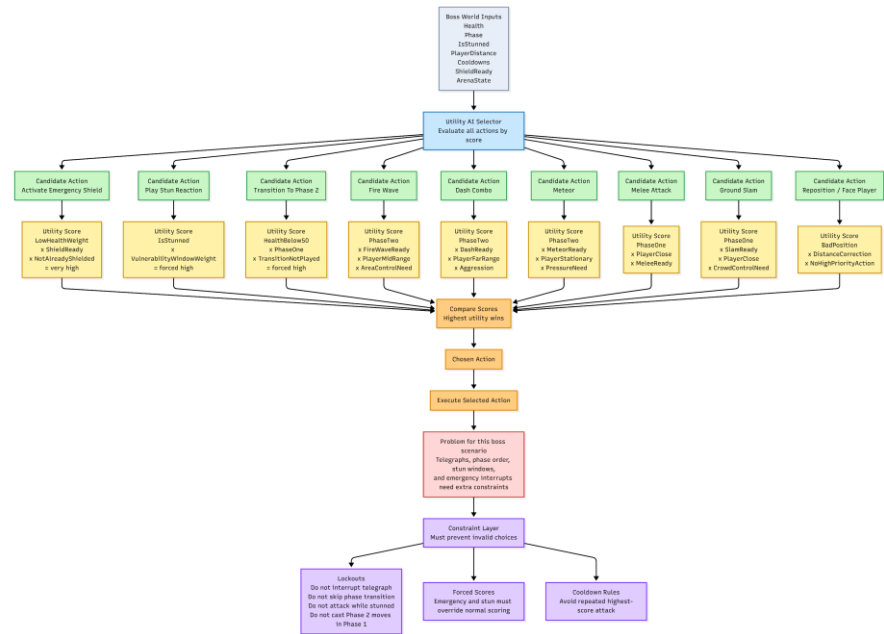
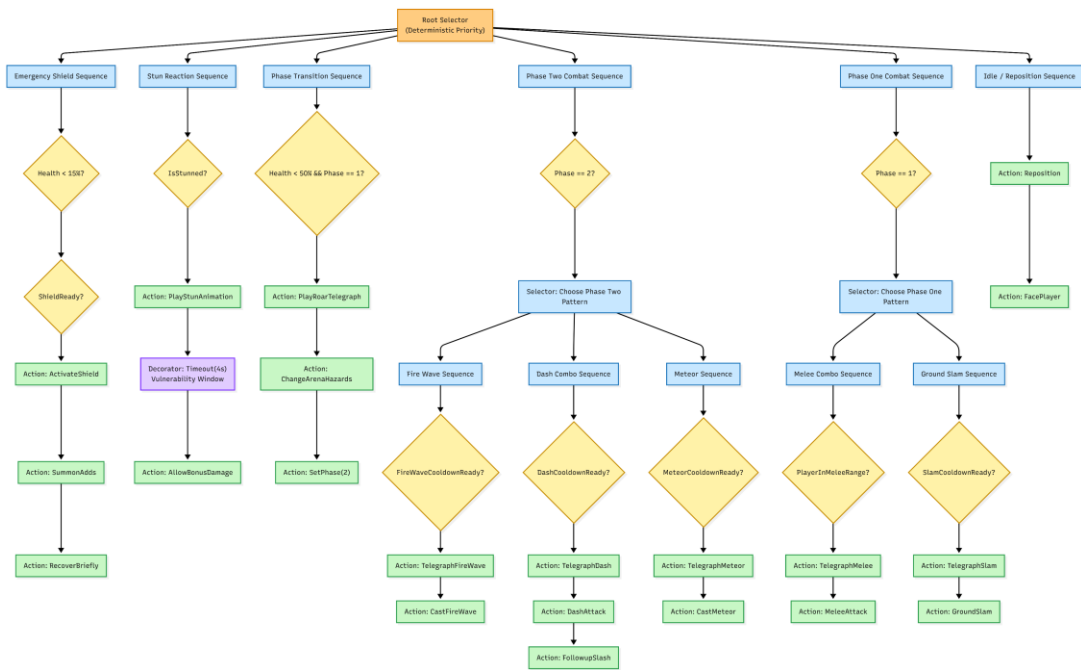


Advantages vs Utility AI

Scenario

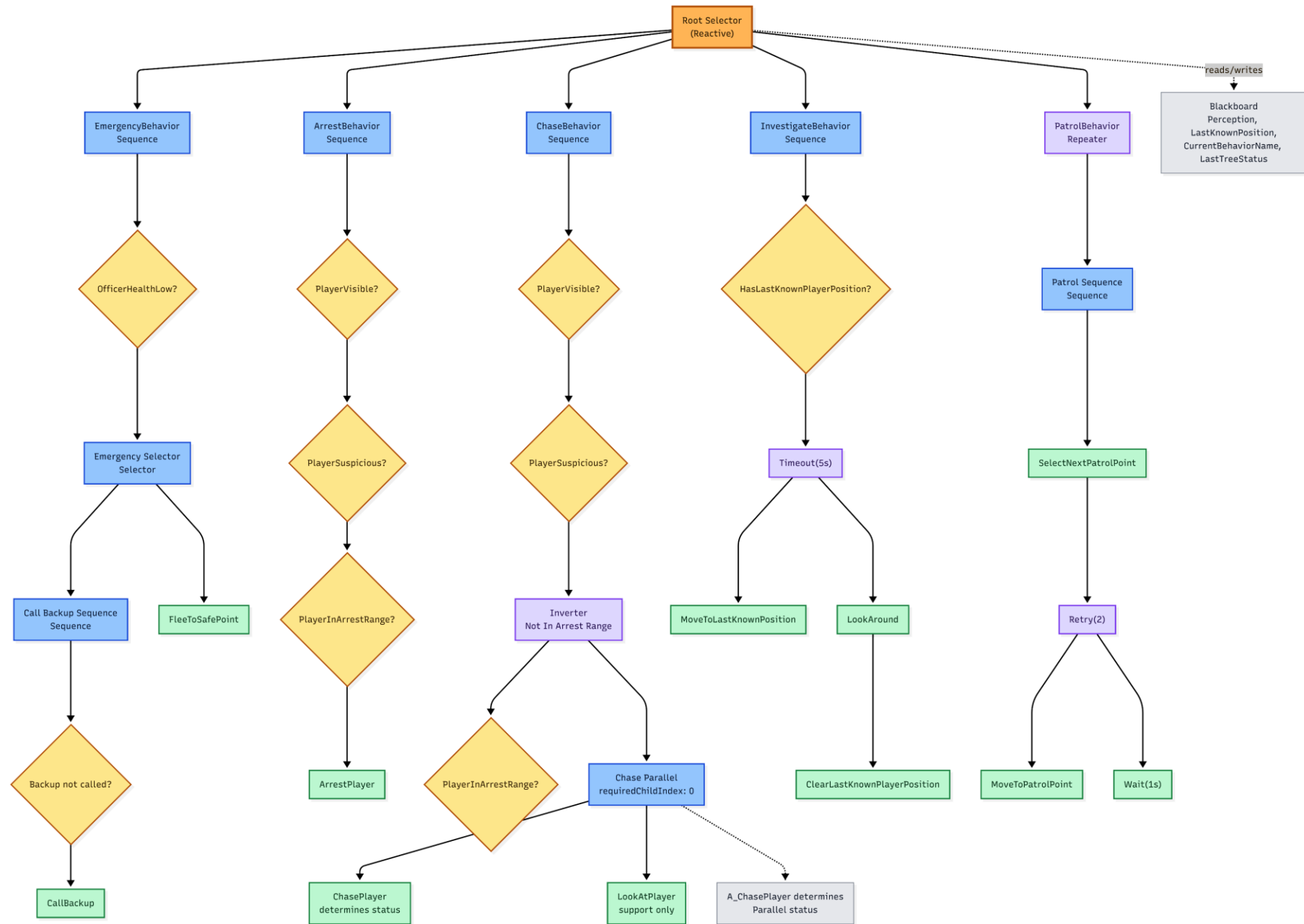
- A boss fight needs clear phases, ordered attack sequences, and high-priority interrupts.
- A Behavior Tree fits because the designer-controlled flow stays readable and predictable.
- Utility AI may become unpredictable unless many extra constraints are added.

Advantages vs Utility AI



Conclusion

- Use Behavior Trees when NPC behavior needs clear priorities, readable structure, and reactive decision-making
- They are especially useful for guards, enemies, bosses, and agents with predictable phases or interrupts
- Behavior Trees work best when behavior should be modular, visualizable, and easier to debug than large FSMs



References

- General Behavior Tree explanation
<https://www.gamedeveloper.com/programming/behavior-trees-for-ai-how-they-work>
- Game Creator Behavior documentation
<https://docs.gamecreator.io/behavior/>
- Spore Behavior Tree documentation
https://chrishecker.com/My_Liner_Notes_for_Spore/Spore_Behavior_Tree_Docs
- Halo 2 GDC Vault talk
<https://www.gdcvault.com/play/1020270/Managing-Complexity-in-the-Halo>
- Halo 2 AI / Behavior DAG source
<https://www.gamedeveloper.com/programming/gdc-2005-proceeding-handling-complexity-in-the-i-halo-2-i-ai>
- Unreal Engine Behavior Tree Node Reference
<https://dev.epicgames.com/documentation/en-us/unreal-engine/behavior-tree-node-reference-in-unreal-engine>
- Unreal Engine Behavior Tree documentation
<https://dev.epicgames.com/documentation/unreal-engine/behavior-trees-in-unreal-engine>
- <https://chatgpt.com/share/6a294274-1178-83ed-bac1-0cfc7fc35a9>